ASSIGNMENT 01

LINEAR DATA STRUCTURES

Q1. Differentiate between Data Structures and ADT

Data Structure	**Abstract Data Type (ADT)**
A data structure is a way of organizing and storing data in memory.	ADT is a logical or mathematical model that defines data and operations on it.
Focuses on **implementation**.	Focuses on **behavior/interface**, not implementation.
Examples: Array, Linked List, Stack, Tree	Examples: Stack ADT, Queue ADT, List ADT
Specifies how data is stored physically.	Specifies what operations are allowed (push, pop, insert, delete).

Q2. Order by Increasing Growth Rate

Slowest → Fastest:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Reasoning :

- **$O(1)$:** Constant time; does not depend on input size.
- **$O(\log n)$:** Grows slowly because input is repeatedly halved.
- **$O(n)$:** Linear growth; time proportional to input.
- **$O(n \log n)$:** Slightly faster than quadratic; common in efficient sorting.
- **$O(n^2)$:** Nested loops cause quadratic growth.
- **$O(2^n)$:** Doubles with each extra input element.
- **$O(n!)$:** Permutation growth; increases extremely fast.

Q3. Time & Space Complexity

3.1 Snippet A (Nested Loops)

Code runs:

```
for i in range(n):
    for j in range(i, n):
```

Summation:

Total iterations

$= (n) + (n-1) + (n-2) + ... + 1$

$= n(n + 1) / 2$

Time Complexity:
$O(n^2)$

Space Complexity:
O(1) (only variable c used)

3.2 Snippet B (Halving Loop)

Each iteration divides the value by 2.

Example:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow ... \rightarrow 1$

Number of steps:

$\log_2 n$

Time Complexity:
O(log n)

Space Complexity:
O(1)

Reason:
The problem size is **halved each iteration**, which is the main idea behind logarithmic complexity.

3.3 Snippet C (Two Phase Program)

Phase 1:

for i in range(n)

Time = O(n)

Phase 2:

```
while j < n:
    j *= 2
```

This doubles each time.

Time = O(log n)

Combined Complexity:

O(n + log n)

Final Complexity:

O(n)

Space Complexity:

O(1)

3.4 Snippet D (Copy + Sort)

b = a[:] → copying list

b.sort()

Copy Time:

O(n)

Sorting Time:

O(n log n)

Total Time Complexity:

O(n log n)

Auxiliary Space Complexity:

O(n)

Reason:

A new list b is created which stores all elements.

Q4. Choosing the Right Paradigm

4.1 Sorting a very large list with guaranteed O(n log n)

Best Paradigm: Divide and Conquer

Reason:

Divide and Conquer splits the problem into smaller subproblems, solves them recursively, and combines the results. Algorithms like Merge Sort use this approach and guarantee O(n log n) worst-case time complexity.

4.2 Minimum coins to make value V

Best Paradigm: Dynamic Programming

Reason:

This problem has overlapping subproblems and optimal substructure. Dynamic Programming stores solutions of smaller values and reuses them to compute the final answer efficiently.

4.3 Maximum number of non-overlapping meetings

Best Paradigm: Greedy Algorithm

Reason:

We select meetings based on earliest finishing time. Greedy works because the locally optimal choice leads to the globally optimal solution.

4.4 Fibonacci for very large n

Best Paradigm: Dynamic Programming

Reason:

Normal recursion repeats calculations many times. Dynamic Programming stores previous Fibonacci values and computes efficiently in O(n).

4.5 Maximum sum subarray

Best Paradigm: Dynamic Programming

Reason:

Kadane's Algorithm is used. It stores the best subarray ending at each position and updates the maximum sum.

Q5. Tower of Hanoi

Python Implementation

```
def hanoi(n, src, aux, dst):
    if n == 1:
        print("Move disk 1 from", src, "to", dst)
        return
    hanoi(n-1, src, dst, aux)
    print("Move disk", n, "from", src, "to", dst)
    hanoi(n-1, aux, src, dst)
```

Move Sequence for n = 3

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Total Number of Moves Formula

Total moves: $2^n - 1$

Example: For n = 3

$2^3 - 1 = 7$ moves

Time Complexity

$O(2^n)$

Reason:

Number of recursive calls doubles for each disk.